

Разработка самовоспроизводящегося компилятора

Балышев Артем Максимович, группа 24.Б72-мм

Научный руководитель: инженер-исследователь кафедры системного программирования, Косарев Д. С.

Санкт-Петербург
2026

Постановка задачи

Целью работы является обеспечение возможности сборки языковых инструментов с помощью `rukamI`

Задачи:

- Расширение стандартной библиотеки:
 - Работа с файловой системой
 - Обработка аргументов командной строки
 - Форматированный вывод
- Реализация манглирования имён и разрешения внешних ссылок
- Реализация простого компилятора и его сборка с помощью `rukamI`

- Все последующие примеры кода — программы, которые компилируются с помощью `rukaml`
- Эти примеры показывают, как написание кода выглядит для пользователя
- Они не имеют отношения к внутреннему устройству `rukaml`

Расширение стандартной библиотеки

Форматированный вывод

```
let rec pp_pexpr oc e =  
  match e with  
  | Pexpr_binop (op, e1, e2) ->  
    fprintf oc "(%a %a %a)" pp_pexpr e1 pp_pbinop op pp_pexpr e2  
  | Pexpr_assign (v, e) ->  
    fprintf oc "%s = %a" v pp_pexpr e  
  | Pexpr_tern (c, t, e) ->  
    fprintf oc "%a ? %a : %a" pp_pexpr c pp_pexpr t pp_pexpr e  
  | ...
```

Обработка аргументов командной строки и работа с файловой системой

```
let main =  
  let input_path, output_path = parse_args sys_argv in  
  let input_channel = open_in input_path in  
  let output_channel = open_out output_path in  
  let () = run_program output_channel (input_all input_channel) in  
  let () = close_in input_channel in  
  let () = close_out output_channel in  
  exit 0
```

- Обеспечение валидности генерируемых ассемблерных меток и предотвращение их коллизий
- Поддержание таблиц для сопоставления ассемблерных меток исходным именам
- Разрешение синонимов для примитивов стандартной библиотеки

Манглирование имён и разрешение внешних ссылок [2/2]

Пример кода, для которого разрешение имён корректно работает

```
(* print is defined somewhere in stdlib *)
```

```
let () = print 42
let () =
  let print _ = () in
  print 42
let () = print 42
let print _ = ()
let () = print 42
let () =
  let print _ = () in
  print 42
let () = print 42
```

Компилятор, собираемый с помощью rukaml [1/3]

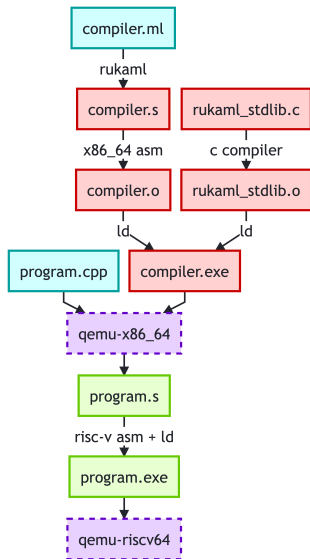
- Создан с целью проверки возможностей rukaml
- Представляет собой простой компилятор подмножества языка C в язык ассемблера RISC-V 64
- Включает в себя:
 - Определение AST
 - Синтаксический анализатор
 - Pretty-printer
 - Модуль кодогенерации
 - Обработку аргументов командной строки, чтение входных данных, вывод результата работы

Компилятор, собираемый с помощью rukaml [2/3]

Пример программы, которую он способен собрать

```
int fact_rec(int n) {  
    if (n < 1) {  
        return 1;  
    }  
    return n * fact_rec(n - 1);  
}  
  
int fact_iter(int n) {  
    int acc = 1;  
    for (int i = 2; i <= n; i = i + 1) {  
        acc = acc * i;  
    }  
    return acc;  
}  
  
int main() {  
    return 144 - fact_rec(4) - fact_iter(5);  
}
```


Компилятор, собираемый с помощью rukaml [3/3]



- `compiler` — компилятор, который собирается с помощью `rukaml`
- `program` — программа, которая собирается с помощью `compiler`

Упрощенная схема архитектуры

- В основном, писались системные тесты, проверяющие работу сразу всех этапов компиляции
- Компилятор, собираемый с помощью `rukaml`, тестировался путём сравнения поведения с `osamlopt`
- Процент покрытия измерялся с помощью утилиты `bisect_ppx`
 - Результат: 79 % по строкам

- Стандартная библиотека расширена ранее упомянутыми примитивами
- Реализовано манглирование имён и разрешение внешних ссылок
- Благодаря внесённым изменениям, с помощью `rukam1` удалось собрать простой компилятор
 - Принципиальных ограничений, не позволяющих собирать с помощью `rukam1` более сложные языковые инструменты, не выявлено
- Выступление с докладом на секционном заседании конференции СПбПУ «Современные технологии в теории и практике программирования»
 - Тезисы доклада включены в сборник материалов конференции, проиндексированный в РИНЦ